Vehicle Management System Database Project

Name - Tarun Badhan

Student ID – 3775318

GitHub Link - https://github.com/tarunbadhan/vehicle-management-database

Abstract

This project is a management system for vehicles. This system manages data related to various vehicles. This project presents a console-based Vehicle Management Database System developed using Java and SQLite. The system was designed to manage vehicle information, owner details, and service records in an organized way. It supports important database operations such as inserting, viewing, updating, and deleting records through a menu-driven Java program connected with JDBC. The project includes an ER diagram, database schema design, sample records, and a working application that interacts with the database. Through this project, I gained practical experience in database design, SQL operations, and Java database connectivity.

Introduction

Database systems are widely used to store, manage, and organize data efficiently. This project focuses on developing a Vehicle Management Database System that allows users to manage vehicles, owners, and service records in a structured manner.

The system is designed to demonstrate the core concepts of database systems, including schema design, relationships between entities, and SQL operations such as insert, select, update, and delete. It uses SQLite as the database and Java with JDBC to interact with the database.

The main objective of this project is to create a simple and functional application that integrates database design with programming. The system provides a menu-driven interface where users can perform different operations such as adding vehicles, updating information, viewing records, and managing service history.

The System

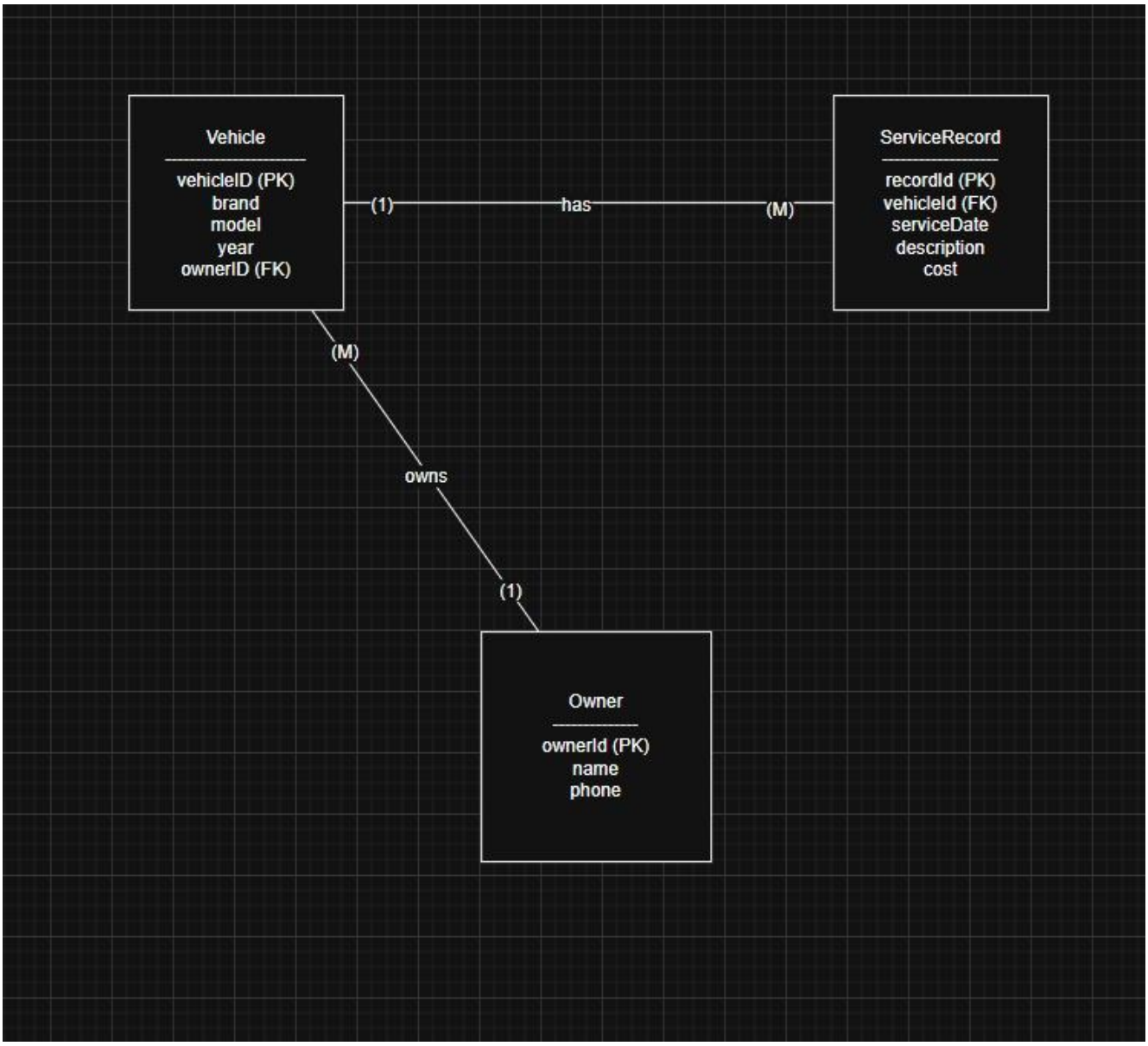
ER Diagram

The Entity-Relationship (ER) diagram represents the structure of the database system. It includes three main entities: Owner, Vehicle, and ServiceRecord.

The Owner entity stores information about vehicle owners. The Vehicle entity stores details about vehicles and is connected to the Owner entity through a one-to-many relationship, where one owner can own multiple vehicles.

The ServiceRecord entity stores service and maintenance information and is connected to the Vehicle entity through another one-to-many relationship, where one vehicle can have multiple service records.

These relationships are implemented in the database using foreign keys to maintain data integrity.



Database Design (DDL)

The database schema is designed based on the ER diagram. It consists of three main tables: Owner, Vehicle, and ServiceRecord. Each table is created using SQL Data Definition Language (DDL) statements, and relationships between tables are implemented using foreign keys.

Owner Table

```
CREATE TABLE Owner (

    owner_id INT PRIMARY KEY,

    full_name TEXT,

    phone TEXT,

    email TEXT,

    address TEXT

);
```

Explanation: The Owner table stores information about vehicle owners. The attribute `owner_id` is the primary key, which uniquely identifies each owner. Other attributes store contact and personal details.

Vehicle Table

```
CREATE TABLE Vehicle (

    vehicle_id INT PRIMARY KEY,

    owner_id INT,

    plate_number TEXT,

    brand TEXT,

    model TEXT,

    year INT,

    color TEXT,

    FOREIGN KEY (owner_id) REFERENCES Owner(owner_id)

);
```

Explanation: The Vehicle table stores details about vehicles. The attribute `vehicle_id` is the primary key. The `owner_id` is a foreign key that connects each vehicle to an owner, establishing a one-to-many relationship between Owner and Vehicle.

ServiceRecord Table

```
CREATE TABLE ServiceRecord (
    service_id INT PRIMARY KEY,
    vehicle_id INT,
    service_date TEXT,
    service_type TEXT,
    cost REAL,
    notes TEXT,
    FOREIGN KEY (vehicle_id) REFERENCES Vehicle(vehicle_id)
);
```

Explanation: The ServiceRecord table stores maintenance and service history of vehicles. The `service_id` is the primary key. The `vehicle_id` is a foreign key that links each service record to a vehicle, representing a one-to-many relationship between Vehicle and ServiceRecord.

Relationships

The database includes two one-to-many relationships. One owner can own multiple vehicles, which is represented by the `owner_id` foreign key in the Vehicle table. Similarly, one vehicle can have multiple service records, which is represented by the `vehicle_id` foreign key in the ServiceRecord table. These relationships ensure proper linkage and data consistency between the tables.

SQL Operations

The system supports the four basic SQL operations: insert, select, update, and delete. These operations are used to manage the records in the database through the Java application.

Insert Operation

The insert operation is used to add new records into the tables. For example, new owners, vehicles, and service records can be inserted into the database.

```
INSERT INTO Owner VALUES (1, 'Tarun', '5061112222', 'tarun@email.com', 'Saint John');

INSERT INTO Vehicle VALUES (101, 1, 'NBX123', 'Toyota', 'Corolla', 2020, 'Black');

INSERT INTO ServiceRecord VALUES (1, 101, '2026-03-01', 'Oil Change', 79.99, 'Done');
```

Explanation: These statements add sample records into the Owner, Vehicle, and ServiceRecord tables. They are useful for creating the initial database instance and testing the system.

Select Operation

The select operation is used to retrieve and display records from the database.

```
SELECT * FROM Vehicle;
SELECT * FROM Owner;
SELECT * FROM ServiceRecord;
```

Explanation: These statements display all records stored in each table. In the Java application, they are used in the menu options for viewing vehicles, owners, and service records.

Update Operation

The update operation is used to modify existing data in the database.

```
UPDATE Vehicle
SET color = 'Red'
WHERE vehicle_id = 101;
```

Explanation: This statement updates the color of a vehicle with a specific vehicle ID. In the application, this operation is used to change vehicle details when needed.

Delete Operation

The delete operation is used to remove records from the database.

```
DELETE FROM Vehicle
WHERE vehicle_id = 101;
```

Explanation: This statement deletes a vehicle record from the Vehicle table. In the application, this is used when a vehicle needs to be removed from the system.

JDBC Implementation

The SQL operations are executed in the Java program using JDBC. The application connects to the SQLite database through the `DatabaseConnection` class and performs database operations using `Statement` and `PreparedStatement`. This allows the user to interact with the database through a menu-driven console program.

Java Application

The system is implemented as a console-based Java application that interacts with the SQLite database using JDBC. The application provides a menu-driven interface that allows users to perform various operations on the database.

The main class contains different methods to handle operations such as viewing vehicles, adding new vehicles, updating vehicle details, deleting vehicles, viewing owners, viewing service records, and adding service records. Each operation corresponds to SQL statements executed through JDBC.

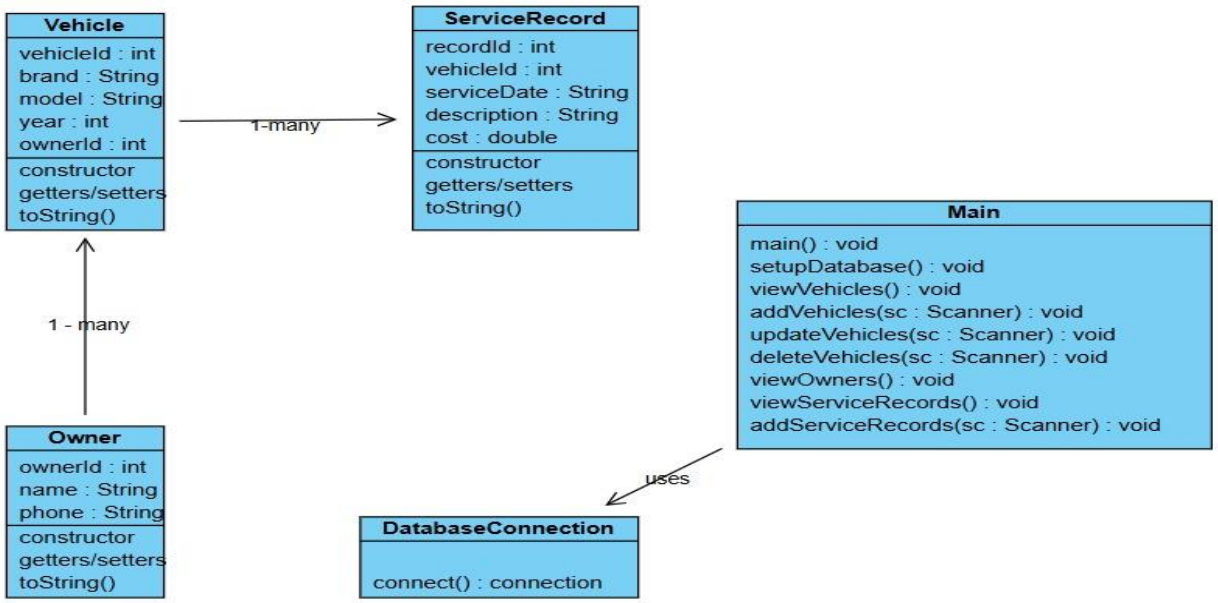
The DatabaseConnection class is used to establish a connection to the database. The application uses both Statement and PreparedStatement to execute SQL queries and updates. Prepared statements are used for inserting and updating data to ensure better handling of user input.

The system allows users to interact with the database in a structured and user-friendly way through simple menu options, demonstrating the integration of database design and application development.

Class Diagram

The class diagram represents the structure of the Java application. It includes the main classes used in the system, such as Main and DatabaseConnection, along with their methods and relationships.

The Main class contains the core logic of the application, including menu handling and database operations such as viewing, adding, updating, and deleting records. The DatabaseConnection class is responsible for establishing a connection to the SQLite database using JDBC. The class diagram reflects how the application interacts with the database and organizes the program into logical components.



Conclusion

This project successfully demonstrates the design and implementation of a Vehicle Management Database System using Java and SQLite. It covers key concepts such as ER diagram design, database schema creation, and SQL operations including insert, select, update, and delete.

Through this project, I gained practical experience in working with databases, designing relationships between entities, and connecting a Java application to a database using JDBC. The system is functional, easy to use, and can be further extended with additional features in the future.

Bibliography

SQLite Documentation. https://www.sqlite.org/docs.html

Oracle Java Documentation (JDBC). https://docs.oracle.com/javase/tutorial/jdbc/